

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Обработка деревьев

Вариант № 3

Студент: **Чернышева А.П.**

Группа: **ИУ7-36Б**

Преподаватель: **Силантьева А. В.**

2025 г.

Описание условия задачи

Построить бинарное дерево поиска, в вершинах которого находятся слова из текстового файла (предполагается, что исходный файл не содержит повторяющихся слов). Вывести его на экран в виде дерева. Удалить все слова, начинающиеся на указанную букву. Сравнить время удаления слов, начинающихся на указанную букву, в дереве и в файле. Реализовать основные операции работы с деревом: обход дерева, включение, исключение и поиск узлов.

Описание ТЗ

Описание исходных данных

Введенный пункт меню.
Имя текстового файла со словами.
Слова для добавления в дерево.
Буква для удаления слов.
Ограничения:
- Максимальная длина слова: MAX_WORD_LENGTH = 100 символов;
- Максимальная длина имени файла: MAX_FILENAME = 100 символов;
- Слова в файле разделены пробельными символами или знаками пунктуации.

Результаты:

Визуальное представление дерева.
Результаты обходов дерева (префиксный, инфиксный, постфиксный).
Статистика удаления слов.
Сравнение времени удаления в дереве и файле.
Экспорт дерева в DOT-формат для визуализации.

Описание задачи, реализуемой программой

Программа реализует систему работы с бинарным деревом поиска с возможностью:
1. Загрузка слов из файла в дерево
2. Добавление слова в дерево
3. Удаление слова из дерева
4. Поиск слова в дереве
5. Удаление слов, начинающихся на указанную букву
6. Обход дерева (префиксный, инфиксный, постфиксный)
7. Визуализация дерева
8. Экспорт в DOT формат
9. Сравнение времени удаления
10. Показать высоту дерева

Способ обращения к программе

Программа запускается через исполняемый файл app.exe.

Описание возможных аварийных ситуаций и ошибок пользователя

1. Ошибка открытия файла
2. Пустой файл или файл без слов
3. Попытка операций с пустым деревом

4. Ввод пустого слова
5. Неверный выбор пункта меню
6. Ошибка выделения памяти

Описание внутренних структур данных

tree_node_t - структура узла дерева

```
typedef struct tree_node {
    char word[MAX_WORD_LENGTH];
    struct tree_node* left;
    struct tree_node* right;
} tree_node_t;
```

char word[MAX_WORD_LENGTH] хранение данных узла - текстового слова
struct tree_node* left указатель на левое поддерево
struct tree_node* right указатель на правое поддерево

Общий алгоритм программы:

1. **Инициализация программы:**
   - Объявление указателя на корень дерева
   - Инициализация переменных для управления состоянием программы
2. **Основной цикл программы:**
   - Вывод меню с доступными операциями
   - Ожидание ввода выбора пользователя
   - Выполнение выбранной операции
3. **Операции меню:**

1: Загрузка слов из файла
2: Добавление слова
3: Удаление слова
4: Поиск слова
5: Удаление по первой букве
6-8: Обходы дерева
9: Визуализация дерева
10: Экспорт в DOT
11: Сравнение времени удаления
12: Высота дерева
0: Выход

Описание алгоритмов функций

tree_node_t *tree_create_node(const char* word) - создание нового узла дерева:

1. Выделение памяти для нового узла с помощью malloc()
2. Проверка успешности выделения памяти
3. Копирование слова в поле word нового узла с защитой от переполнения
4. Инициализация указателей left и right значением NULL

5. Возврат указателя на созданный узел

tree_node_t* tree_insert(tree_node_t* root, const char* word) - добавление узла в дерево:

1. Если текущий узел (root) равен NULL - вызов tree_create_node() и возврат нового узла
2. Сравнение добавляемого слова со словом в текущем узле с помощью strcmp()
3. Если слово меньше слова в текущем узле - рекурсивный вызов для левого поддерева
4. Если слово больше слова в текущем узле - рекурсивный вызов для правого поддерева
5. Если слова равны - игнорирование добавления (дерево не содержит дубликатов)
6. Возврат текущего узла после выполнения рекурсивных вызовов

tree_node_t* tree_search(tree_node_t* root, const char* word) - поиск узла в дереве:

1. Если текущий узел равен NULL или слова совпадают - возврат текущего узла
2. Сравнение искомого слова со словом в текущем узле
3. Если слово меньше - рекурсивный поиск в левом поддереве
4. Если слово больше - рекурсивный поиск в правом поддереве

tree_node_t* tree_min_node(tree_node_t* root) - поиск минимального узла:

1. Проверка, что текущий узел не NULL
2. Последовательный переход по левым указателям до достижения NULL
3. Возврат самого левого узла в поддереве

tree_node_t* tree_max_node(tree_node_t* root) - поиск максимального узла:

1. Проверка, что текущий узел не NULL
2. Последовательный переход по правым указателям до достижения NULL
3. Возврат самого правого узла в поддереве

tree_node_t* tree_delete(tree_node_t* root, const char* word)- удаление узла из дерева:

1) Поиск узла для удаления
- Сравнение искомого слова со словом в текущем узле
- Рекурсивный поиск в левом поддереве при меньшем значении
- Рекурсивный поиск в правом поддереве при большем значении

2)Когда узел найден:

Если узел имеет только правого потомка или не имеет потомков
- Сохранение правого потомка во временную переменную
- Освобождение памяти текущего узла
- Возвращение правого потомка в качестве нового корня поддерева

Если узел имеет только левого потомка
- Сохранение левого потомка во временную переменную
- Освобождение памяти текущего узла
- Возвращение левого потомка в качестве нового корня поддерева

Если узел имеет обоих потомков
- Поиск минимального узла в правом поддереве
- Запоминание родителя минимального узла
- При отсутствии прямого правого потомка:
- Перенаправление правого поддерева минимального узла к левому указателю его родителя
- Установление связи между минимальным узлом и правым поддеревом удаляемого узла
- Соединение минимального узла с левым поддеревом удаляемого узла
- Освобождение памяти удаляемого узла
- Возвращение минимального узла в качестве нового корня поддерева
3) Возврат обновленного указателя на корень поддерева
tree_node_t* tree_delete_by_letter(tree_node_t* root, char letter, int* deleted_count) - удаление по первой букве:

1. Если текущий узел равен NULL - возврат NULL
2. Рекурсивный вызов для левого и правого поддеревьев
3. Проверка первой буквы слова в текущем узле:
   Если первая буква совпадает с заданной - увеличение счетчика удаленных и вызов tree_delete() для текущего узла
4. Возврат обновленного указателя на корень

void tree_preorder_traversal(tree_node_t* root) - префиксный обход:

1. Если узел равен NULL - возврат
2. Вывод слова текущего узла
3. Рекурсивный вызов для левого поддерева
4. Рекурсивный вызов для правого поддерева

void tree_inorder_traversal(tree_node_t* root) - инфиксный обход:

1. Если узел равен NULL - возврат
2. Рекурсивный вызов для левого поддерева
3. Вывод слова текущего узла
4. Рекурсивный вызов для правого поддерева

void tree_postorder_traversal(tree_node_t *root) - постфиксный обход:

1. Если узел равен NULL - возврат
2. Рекурсивный вызов для левого поддерева
3. Рекурсивный вызов для правого поддерева
4. Вывод слова текущего узла

void tree_print_visual(tree_node_t* root, int level) - визуализация дерева:

1. Если узел равен NULL - возврат
2. Рекурсивный вызов для правого поддерева с уровнем level+1
3. Вывод отступов (4 пробела на каждый уровень)
4. Вывод слова текущего узла
5. Рекурсивный вызов для левого поддерева с уровнем level+1

int tree_height(tree_node_t* root) - вычисление высоты дерева:

1. Если узел равен NULL - возврат 0
2. Рекурсивное вычисление высоты левого поддерева

3. Рекурсивное вычисление высоты правого поддерева
4. Возврат максимальной высоты плюс 1 (текущий узел)

void tree_free(tree_node_t *root) - освобождение памяти дерева:

1. Если узел равен NULL - возврат
2. Рекурсивный вызов для левого поддерева
3. Рекурсивный вызов для правого поддерева
4. Освобождение памяти текущего узла

int tree_read_from_file(const char* filename, tree_node_t** root) - загрузка из файла:

1. Открытие файла для чтения
2. Проверка успешности открытия
3. Построчное чтение файла в буфер
4. Разбиение строки на слова с помощью strtok()
5. Приведение слов к нижнему регистру
6. Добавление каждого слова в дерево с помощью tree_insert()
7. Закрытие файла
8. Возврат статуса операции

double tree_measure_deletion_time(tree_node_t** root, char letter) - замер времени удаления:

1. Запись начального времени с помощью clock()
2. Вызов tree_delete_by_letter() для удаления слов по букве
3. Запись конечного времени
4. Вычисление разницы и преобразование в микросекунды
5. Возврат времени выполнения

double file_measure_deletion_time(const char* filename, char letter) - замер времени удаления в файле:

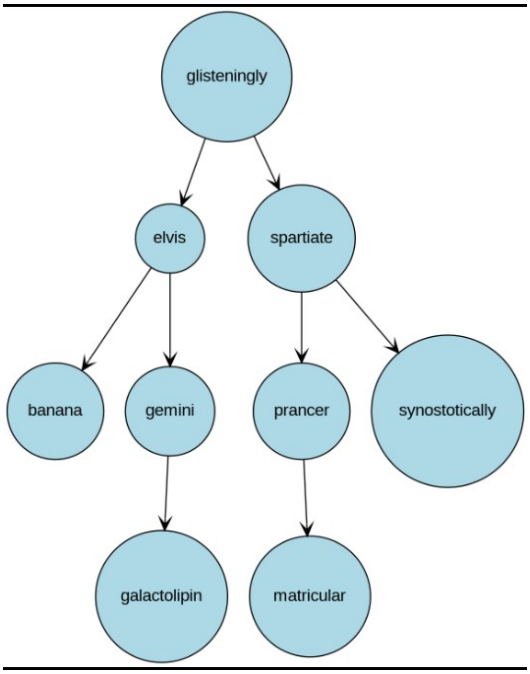
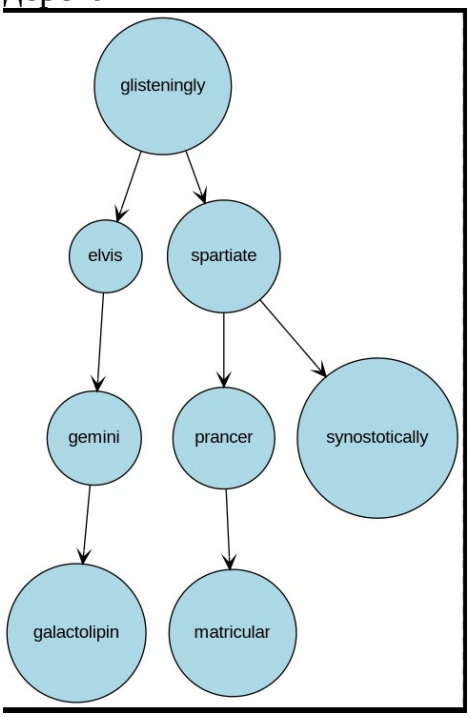
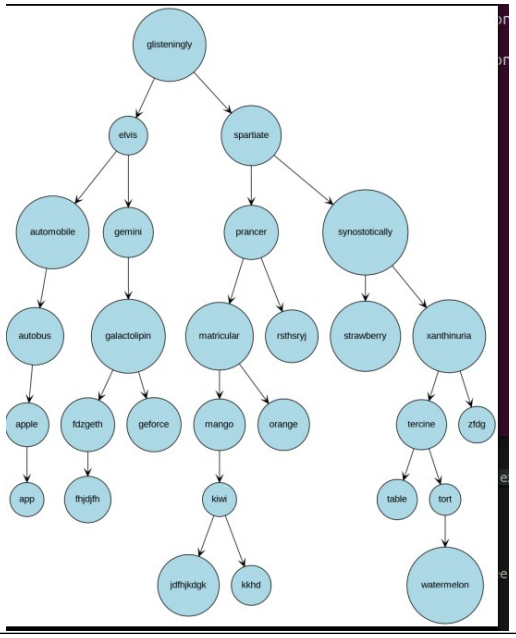
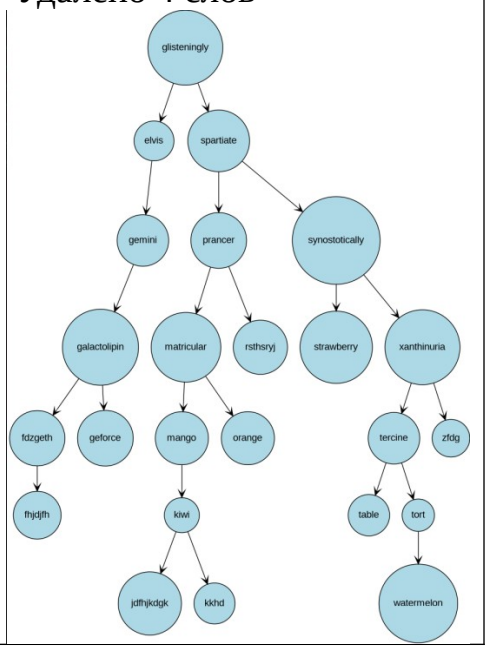
1. Открытие исходного файла и временного файла
2. Запись начального времени
3. Построчное чтение исходного файла
4. Запись во временный файл только тех слов, которые не начинаются на заданную букву
5. Запись конечного времени
6. Закрытие файлов, замена исходного файла временным
7. Вычисление и возврат времени выполнения

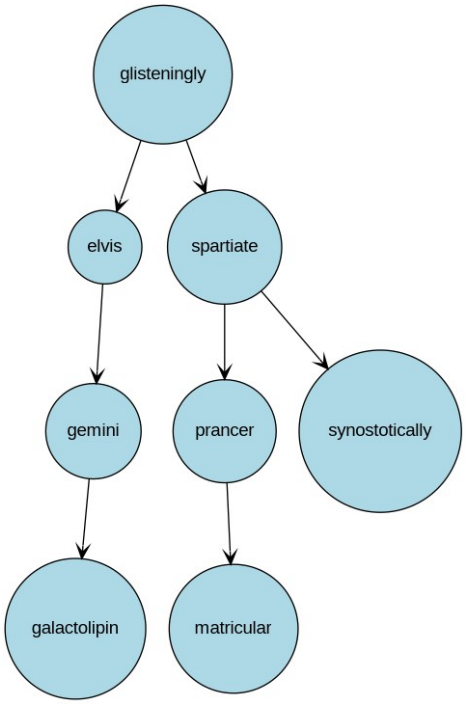
void tree_export_to_dot(tree_node_t* root, const char* output_filename) - экспорт в DOT:

1. Проверка, что дерево не пустое
2. Создание DOT-файла
3. Запись заголовка DOT-формата
4. Рекурсивная генерация связей между узлами с помощью tree_generate_dot()
5. Запись заключительной части файла
6. Закрытие файла

Тесты

Позитивные тесты

Номер теста	Описание	Ожидаемый результат
1	Добавление слов из корректного файла 1.txt	Дерево успешно построено из файла 1.txt
2	Добавление нового слова "xylophone"	Слово 'xylophone' добавлено в дерево
3	Удаление существующего слова "banana" 	Слово 'banana' удалено из дерева 
4	Поиск существующего слова 'banana'	Слово 'banana' найдено в дереве
5	Удаление слов по букве 'a' 	Удалено 4 слов 

6	Показать высоту дерева	Высота дерева: 15
7	Сравнение времени удаления на букву 'c'	Анализ эффективности
8	Префиксный обхорд	Префиксный обход: glisteningly elvis gemini galactolipin spartiate prancer matricular synostotically
9	Инфиксный обход	Инфиксный обход: elvis galactolipin gemini glisteningly matricular prancer spartiate synostotically
10	Постфиксный обход	Постфиксный обход: galactolipin gemini elvis matricular prancer synostotically spartiate glisteningly
11	Визуализация дерева	

Негативные тесты

Номер теста	Описание	Ожидаемый результат
1	Загрузка несуществующего файла	Ошибка загрузки файла
2	Файл без слов	Файл пуст или не содержит слов Ошибка загрузки файла
3	Ввод пустой строки	Ошибка: пустое слово

5	Удаление из пустого дерева	Дерево пусто
6	Поиск в пустом дереве	Дерево пусто
7	Удаление несуществующего слова	Слово 'a' не найдено в дереве
8	Ввод числа вне диапазона меню	Неверный выбор
9	Некорректный ввод в меню (например ааа)	Ошибка ввода
10	Любой обход при пустом дереве	Дерево пусто
11	Удаление по букве в пустом дереве	Дерево пусто
12	Удаление по букве, ни одно слово с которой не начинается	Не нашлось таких слов

Оценка эффективности

Если по указателю 100 итераций:

Размер	Время удаления в дереве, мкс	Время удаления в файле, мкс	Эффективность дерева относительно файла
100	2.2	43.8	19.91
500	11.8	168.9	13.3
1000	23.4	279.8	11.96

Если копировать данные 100 итераций:

Размер	Время удаления в дереве, мкс	Время удаления в файле, мкс	Эффективность дерева относительно файла
100	2.8	36.4	13
500	18.4	165.1	8.97
1000	25.3	279.8	11.06

Вывод

Использование бинарного дерева поиска для операции удаления данных значительно эффективнее по сравнению с обработкой данных в файле напрямую. При работе по указателю ускорение достигает от 12 до 20 раз в зависимости от размера данных, а при копировании данных — от 9 до 13 раз. Наилучшая эффективность наблюдается при небольших размерах данных (100 элементов), где дерево работает почти в 20 раз быстрее файлового подхода. При сравнении двух методов работы с бинарным деревом - копирования данных и перестановки указателей видно, что

метод с указателями в большинстве случаев быстрее, особенно на малых и средних объёмах данных.100 элементов: метод с указателями быстрее на 53%. 500 элементов: метод с указателями быстрее на 59%. 1000 элементов: метод с указателями быстрее на 8%.

Контрольные вопросы

1. Что такое дерево?
   Дерево - это рекурсивная структура данных, состоящая из узлов, где каждый узел имеет значение и ссылки на дочерние узлы. Бинарное дерево - дерево, где каждый узел имеет не более двух потомков.
2. Каким образом и сколько памяти выделяется под хранение дерева?
   Память выделяется динамически для каждого узла. Каждый узел содержит данные и указатели на потомков. Общий объем памяти зависит от количества узлов.
3. Каким образом освобождается память при удалении элемента дерева?
   При удалении узла память освобождается явно с помощью free(). Для полного удаления дерева используется рекурсивный обход с освобождением памяти всех узлов.
4. Что такое дерево двоичного поиска?
   Это бинарное дерево, в котором для каждого узла выполняется: все элементы левого поддерева меньше корня, все элементы правого поддерева больше корня.
5. Каким образом эффективнее реализовывать дерево?
   Эффективность зависит от сбалансированности дерева. Сбалансированные деревья (AVL, красно-черные) обеспечивают логарифмическое время операций, в то время как вырожденные деревья вырождаются в линейные списки.